How to Use This Coding Guide

The assessment includes Python programming questions (candidate reports mention ~2-3 coding problems).

These analyst/data-science problems emphasise counting, aggregation and data manipulation. Every solution is executed and verified.

Authenticity labels appear beside each problem.

Problems

Problem 1: Most Frequent Word

Difficulty: ***Easy*** | *Pattern:* ***Hash Map*** | *Authenticity:* ***Community Reported***

Problem Statement

Return the most frequent word in a sentence; ties resolved by first appearance; case-insensitive.

Input Format

A single line sentence.

Output Format

The most frequent word (lowercase).

Constraints

Words separated by single spaces.

Sample Input

```
the cat sat on the mat the cat
```

Sample Output

```
the
```

Python Solution

```
def most_frequent_word(sentence):
    counts = {}
    best_word, best = None, 0
    for w in sentence.lower().split():
        counts[w] = counts.get(w, 0) + 1
        if counts[w] > best:
            best, best_word = counts[w], w
    return best_word

print(most_frequent_word('the cat sat on the mat the cat'))
```

Step-by-Step Explanation

Count words; keep the best, updating only on a strictly higher count so ties keep the first word. 'the' occurs 3 times.

Dry Run

```
the->1(best);cat->1;the->2;the->3 -> the
```

Time Complexity: **O(N)** | Space Complexity: **O(K)**

Problem 2: Find the Missing Number

Difficulty: ***Easy*** | *Pattern:* ***Math / Summation*** | *Authenticity:* ***Community Reported***

Problem Statement

An array holds n distinct numbers from 0..n with exactly one missing. Return it.

Input Format

Line 1: n. Line 2: the n numbers.

Output Format

The missing number.

Constraints

1 <= n <= 10^6

Sample Input

```
8
3 0 1 5 2 7 6 8
```

Sample Output

```
4
```

Python Solution

```
def missing_number(nums):
    n = len(nums)
    return n * (n + 1) // 2 - sum(nums)

print(missing_number([3, 0, 1, 5, 2, 7, 6, 8]))
```

Step-by-Step Explanation

Sum of 0..n minus the actual sum gives the missing value. 36 - 32 = 4.

Dry Run

```
expected 36, actual 32 -> 4
```

Time Complexity: **O(N)** | Space Complexity: **O(1)**

Problem 3: FizzBuzz

Difficulty: ***Easy*** | *Pattern:* ***Simulation / Modulo*** | *Authenticity:* ***Frequently Repeated***

Problem Statement

Print 1..n; multiples of 3 -> Fizz, of 5 -> Buzz, of both -> FizzBuzz. Space-separated.

Input Format

A single integer n.

Output Format

The FizzBuzz sequence on one line.

Constraints

1 <= n <= 10^4

Sample Input

```
15
```

Sample Output

```
1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz
```

Python Solution

```
def fizzbuzz(n):
    out = []
    for i in range(1, n + 1):
        if i % 15 == 0:
            out.append('FizzBuzz')
        elif i % 3 == 0:
            out.append('Fizz')
        elif i % 5 == 0:
            out.append('Buzz')
        else:
            out.append(str(i))
    return ' '.join(out)

print(fizzbuzz(15))
```

Step-by-Step Explanation

Check 15 first, then 3, then 5, else the number. A very common warm-up coding question.

Dry Run

```
3->Fizz,5->Buzz,15->FizzBuzz
```

Time Complexity: **O(N)** | Space Complexity: **O(N)**

Problem 4: Two Sum

Difficulty: ***Easy-Medium*** | *Pattern:* ***Hash Map*** | *Authenticity:* ***Frequently Repeated***

Problem Statement

Return indices of the two numbers that sum to the target (exactly one solution).

Input Format

Line 1: array. Line 2: target.

Output Format

Two indices in increasing order.

Constraints

2 <= n <= 10^5

Sample Input

```
2 7 11 15
```

```
9
```

Sample Output

```
0 1
```

Python Solution

```
def two_sum(nums, target):
    seen = {}
    for i, x in enumerate(nums):
        if target - x in seen:
            return (seen[target - x], i)
        seen[x] = i

i, j = two_sum([2, 7, 11, 15], 9)
print(i, j)
```

Step-by-Step Explanation

Store indices; when a complement is found return the pair. 2 + 7 = 9.

Dry Run

```
at 7, complement 2 seen -> (0,1)
```

Time Complexity: **O(N)** | Space Complexity: **O(N)**

Problem 5: Find Duplicates in a List

Difficulty: ***Easy*** | *Pattern:* ***Counting*** | *Authenticity:* ***Community Reported***

Problem Statement

Return values appearing more than once, ascending, space-separated; 'none' if none.

Input Format

Line 1: n. Line 2: n integers.

Output Format

Duplicates ascending, or 'none'.

Constraints

1 <= n <= 10^5

Sample Input

```
8
4 3 2 7 8 2 3 1
```

Sample Output

```
2 3
```

Python Solution

```
def find_duplicates(nums):
    counts = {}
    for x in nums:
        counts[x] = counts.get(x, 0) + 1
    dups = sorted(v for v, c in counts.items() if c > 1)
    return ' '.join(map(str, dups)) if dups else 'none'

print(find_duplicates([4, 3, 2, 7, 8, 2, 3, 1]))
```

Step-by-Step Explanation

Count, then keep values with count>1 and sort. 2 and 3 repeat.

Dry Run

```
2->2,3->2 -> [2,3]
```

Time Complexity: **O(N log N)** | Space Complexity: **O(N)**

Problem 6: Reverse the Words in a Sentence

Difficulty: ***Easy*** | *Pattern:* ***String*** | *Authenticity:* ***Community Reported***

Problem Statement

Reverse the order of the words in a sentence.

Input Format

A single line sentence.

Output Format

Reversed word order.

Constraints

1 <= length <= 10^4

Sample Input

```
data science is fun
```

Sample Output

```
fun is science data
```

Python Solution

```
def reverse_words(s):
    return ' '.join(reversed(s.split()))

print(reverse_words('data science is fun'))
```

Step-by-Step Explanation

split, reverse, join.

Dry Run

```
['data','science','is','fun'] reversed
```

Time Complexity: **O(N)** | Space Complexity: **O(N)**

Problem 7: Second Largest Unique Value

Difficulty: ***Easy-Medium*** | *Pattern:* ***Set + Sorting*** | *Authenticity:* ***Community Reported***

Problem Statement

Return the second largest distinct value, or -1 if it does not exist.

Input Format

Line 1: n. Line 2: n integers.

Output Format

Second largest distinct value or -1.

Constraints

1 <= n <= 10^5

Sample Input

```
6
10 10 9 8 8 7
```

Sample Output

```
9
```

Python Solution

```
def second_largest_unique(nums):
    d = sorted(set(nums), reverse=True)
    return d[1] if len(d) >= 2 else -1

print(second_largest_unique([10, 10, 9, 8, 8, 7]))
```

Step-by-Step Explanation

Dedupe, sort descending, take index 1. Distinct desc [10,9,8,7] -> 9.

Dry Run

```
set->{7,8,9,10}; [10,9,8,7][1]=9
```

Time Complexity: **O(N log N)** | Space Complexity: **O(N)**

Problem 8: Group Frequencies (value_counts)

Difficulty: ***Medium*** | *Pattern:* ***Aggregation*** | *Authenticity:* ***Research-Backed Company Style***

Problem Statement

Output each distinct label with its count, sorted by count desc then label asc (like pandas value_counts).

Input Format

Line 1: n. Line 2: n labels.

Output Format

One 'label:count' per line.

Constraints

1 <= n <= 10^5

Sample Input

```
7
apple banana apple cherry banana apple cherry
```

Sample Output

```
apple:3
banana:2
```

```
cherry:2
```

Python Solution

```
def value_counts(labels):
    counts = {}
    for label in labels:
        counts[label] = counts.get(label, 0) + 1
    ordered = sorted(counts.items(), key=lambda kv: (-kv[1], kv[0]))
    return '\n'.join(f'{k}:{v}' for k, v in ordered)

data = ['apple', 'banana', 'apple', 'cherry', 'banana', 'apple', 'cherry']
print(value_counts(data))
```

Step-by-Step Explanation

Count, then sort by (-count, label). Core analytics frequency table.

Dry Run

```
apple3,banana2,cherry2
```

Time Complexity: **O(N log N)** | Space Complexity: **O(K)**

Problem 9: Count Word Frequency Above Threshold

Difficulty: ***Medium*** | *Pattern:* ***Hash Map / Filter*** | *Authenticity:* ***Research-Backed Company Style***

Problem Statement

Given words and a threshold t, count how many distinct words occur at least t times.

Input Format

Line 1: t. Line 2: the words.

Output Format

Number of distinct words with count >= t.

Constraints

1 <= words <= 10^5

Sample Input

```
2
a b a c b a d b
```

Sample Output

```
2
```

Python Solution

```
def frequent_words(words, t):
    counts = {}
    for w in words:
        counts[w] = counts.get(w, 0) + 1
    return sum(1 for c in counts.values() if c >= t)

print(frequent_words(['a','b','a','c','b','a','d','b'], 2))
```

Step-by-Step Explanation

Count words; a=3, b=3 meet threshold 2 (c=1,d=1 do not) -> 2.

Dry Run

```
a3,b3,c1,d1; >=2 -> a,b -> 2
```

Time Complexity: **O(N)** | Space Complexity: **O(K)**

Tredence

Python Coding Problems

Applied Role(s):

Analyst - Data Scientist

Online Assessment Preparation Material

Version 1.1 • Last Updated: July 2026

Every question carries an authenticity label. See the legend on the next page.

Table of Contents

Authenticity Legend & How to Read This Document

Every question and problem in this document is labelled with an **authenticity classification** so you know how closely it maps to what candidates have actually reported. Labels are assigned conservatively based on research; authored practice questions are never presented as real exam questions.

Label	Meaning
Verified Previous-Year	The specific problem/pattern is documented from actual reported assessment experiences.
Frequently Repeated	The topic or pattern type is documented as recurring across multiple hiring cycles.
Community Reported	A widely shared, role-relevant pattern reported across placement communities and forums.
Research-Backed Company Style	Authored to match the researched difficulty, style and pattern - practice, not a real past question.

The correct option in every MCQ is highlighted in green and also restated below the question. Study the pattern, not just the answer.